

1. What does “salting” mean in the context of password hashing, and why is it used in Spring Security?

A: **Salting** means adding a unique, random string of data to a user's password *before* it is run through the hashing function. It is used to protect against **Rainbow Table attacks** and dictionary attacks. Because each user has a unique salt, even if two users choose the exact same password, their final hashed values will be different, making precomputed attacks ineffective.

2. What kind of information is typically stored inside a JWT (JSON Web Token)?

A: A **JWT** typically stores **claims**, which are pieces of information about the user or session. These include required claims like **iss** (issuer), **exp** (expiration time), and **sub** (subject, usually the user ID). It also often stores authorization claims, such as the user's **roles** or specific permissions. It is crucial never to store sensitive information like passwords in the payload because it is only signed, not encrypted.

3. What is SLF4J, and why is it commonly used for logging in Spring Boot applications?

A: **SLF4J (Simple Logging Facade for Java)** is not a logging framework itself; it is an **abstraction layer** or *façade*. It is commonly used because it decouples the application code from the logging implementation. This means you can write standard logging calls, and if you later decide to switch from the default **Logback** to a different system like Log4j2, you don't need to change your application's source code.